

Assignment # 01

Quality Evaluation of AI-Generated Software

Course Code	SE3002	Total Marks	100
Team Size	Pair (2 students)	Due Date	_____
Project Selection	Section tab — first come, first served	Submission	Report + code + tool evidence

Purpose of the Assignment

Core purpose: This assignment evaluates your defensible decision-making. You must be able to explain why you selected particular requirements, assumptions, tests, expected results, defects, and quality conclusions. Running the required tools is compulsory, but raw tool output or a high score is not sufficient without your own interpretation and evidence.

Tools and Technology

- **Development technology:** There is no restriction on programming language, frontend, backend, framework, or database. The completed application must run and implement the selected requirements.
- **AI-assisted coding:** An AI-assisted coding/vibe-coding tool may be used. The pair remains responsible for the code and must record important assumptions introduced during development.
- **SonarQube (required):** After the selected scope has been fully implemented and the baseline code has been frozen, the pair must run the complete codebase through SonarQube and submit the generated report/evidence. Do not scan only selected files or snippets.
- **Jira (required):** Use Jira to record confirmed, reproducible software defects found through testing.

Official setup and tutorial links: [SonarQube Community Build download](#) | [SonarQube quick-start tutorial](#) | [Jira Free](#) | [Create a Jira work item](#)

Team and Project Selection

- Complete the assignment in pairs (two students).
- Cross section formulation of groups is not allowed.
- Enter both students' names/IDs in the project row on the Google Sheet tab assigned to your section.
- Choose one of the 45 supplied SRSs by using a row marked Available. Within your section, each SRS may be assigned to only one pair; the same SRS may be chosen independently in another section. A complete entry in your section tab confirms the first-come, first-served selection.

Requirement Selection Rules

- Select exactly 10 requirements from the chosen SRS: 7 Functional Requirements (70%) and 3 Non-Functional Requirements (30%). Preserve the original requirement IDs and wording where possible.
- No more than 3 of the 7 FRs may be pure CRUD requirements.
- Complete management of one entity counts as one FR. For example, create, view, edit, and delete a student record together count as one 'Manage Student Record' FR—not four separate FRs.
- The remaining FRs should represent meaningful behaviour such as business rules, validation, calculations, decisions, workflows, permissions, or interactions between features.

Required Workflow

1. **Register the pair and project.** Use only your section's tab and complete the row for an Available SRS before another pair in the same section does.
2. **Select the scope.** Choose 7 FRs and 3 NFRs using the rules above.
3. **Develop the application.** Use an acceptable AI-assisted approach and implement the selected scope with an observable GUI.
4. **Freeze the baseline.** Preserve the first complete runnable version that will be evaluated.
5. **Run SonarQube.** Submit the complete frozen codebase to SonarQube and retain the generated report/evidence.
6. **Design and execute tests.** Test all selected FRs, including manual system-level cases.
7. **Investigate and report defects.** Confirm reproducible implementation defects before entering them in Jira.
8. **Conclude and defend.** Use the collected evidence to reach a scoped quality judgment and explain all decisions during the demo/viva.

Assignment Tasks

Part 1 — Requirement Scope and AI Assumptions (30 marks)

Identify the selected SRS and record exactly 10 requirements. Your table must visibly contain 7 FR rows and 3 NFR rows. For each requirement, explain why it was selected, any implementation risk, and any material assumption introduced by the AI-assisted development process.

Classify the basis for each assumption as: (a) supported by the SRS, (b) justified by an explicit design decision, or (c) unsupported. Unsupported assumptions must not be hidden.

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
R__	FR / NFR	...	...	...	SRS / design decision / unsupported

Part 2 — AI-Generated GUI Baseline (10 marks)

Implement the selected 10-requirement scope as a runnable application. The GUI must expose the selected functional behaviours and display observable success/error feedback. Visual polish is secondary to correct, testable behaviour.

- Freeze the first complete runnable implementation before quality evaluation.
- Keep the baseline unchanged while collecting SonarQube and test evidence. If you later fix defects, preserve the evaluated baseline separately.
- Submit the baseline source code, setup/run instructions, and a brief record of the AI-assisted development approach and major assumptions.

Part 3 — Quality Evaluation (45 marks)

A. SonarQube Report and NFR Evaluation (15 marks)

Required SonarQube action: Students must run the complete frozen codebase through SonarQube themselves and submit the report/evidence generated for that codebase. This is required for every pair. The task is not satisfied by describing SonarQube, copying sample output, or scanning only selected code.

From the complete SonarQube output, select approximately five meaningful findings and interpret them. For each selected finding, identify what SonarQube reported, where it occurs, why it matters, and what action the evidence supports. Merely listing ratings, issue counts, or the Quality Gate status is not interpretation.

The SonarQube report and the evaluation of the 3 selected NFRs are both required. Use SonarQube evidence for an NFR only when the reported measure is relevant. For runtime or user-facing NFRs—such as performance, usability, availability, or scalability—use an appropriate measurement, targeted test, or structured inspection. If the SRS gives no defensible threshold, report that limitation instead of inventing one.

SonarQube quality area	Examples of report evidence to interpret
Maintainability	Maintainability issues/rating, code smells, duplicated code, complexity, remediation effort
Reliability	Reliability issues/bugs and reliability rating
Security	Security issues/vulnerabilities, security rating, and security hotspots
Context	Lines of code, coverage if configured, and Quality Gate status

Use one row for each selected NFR:

NFR	SonarQube evidence (if relevant)	Other evaluation method	Finding / judgment	Limitation
NFR__	... / N/A	... / N/A	...	...

B. Functional Test Derivation, Execution and Traceability (30 marks)

Derive test conditions for all 7 FRs, then design and execute 12–15 test cases. The full set must include normal behaviour, business rules, exact boundaries, and invalid/error conditions. Expected results must come from the SRS or another stated, defensible test basis.

Minimum test-set requirements:

- At least 2 test cases must target exact boundary values.
- At least 2 test cases must target invalid input or error handling.
- At least 3 test cases must be system-level tests executed manually by a person. Automation is not expected because a system-test automation tool has not been covered. A system-level case exercises the integrated application from a user-visible entry point to an observable end result; it is not a single function or isolated component test.
- In total, include at least 2 non-trivial/complex test cases whose genuine execution status is FAILED or BLOCKED. These may be two FAILED cases, two BLOCKED cases, or one of each. Do not manufacture an outcome or relabel a PASSED case merely to meet this requirement.
- A FAILED test was executed but the actual result did not match the expected result. A BLOCKED test could not be completed because a required precondition, dependency, environment, or earlier step prevented execution. Record the exact reason and evidence for either status.

The same test may satisfy more than one category—for example, a manual system-level test may also cover an exact boundary—but you must label the relevant categories clearly.

Required Test Records

Table A — Test Condition Record

Test basis / requirement	Condition ID	Test condition
R__	COND-__	...

Table B — Test Case Record (repeat for each case)

ID / title	Unique ID and purpose
Level / category	System/manual, boundary, invalid/error, normal, etc.
Test basis / objective	Requirement or rule being checked and why
Preconditions	What must already be true
Test data	Concrete inputs and boundary values
Steps	Actions performed
Expected result	Defensible result from the stated test basis
Actual result	What happened during execution
Status	PASSED / FAILED / BLOCKED / NOT EXECUTED
Evidence	Screenshot, log, response, report, or other proof

Table C — Traceability Record

Requirement	Condition	Test case	Execution result	Defect report
R__	COND-__	TC-__	PASSED / FAILED / BLOCKED	BUG-__ / N/A

Status Examples

Example	Correct status and reasoning
Capacity boundary	FAILED — At enrolled = capacity, the SRS requires rejection, but the application accepts registration. The case was executed and actual ≠ expected.
End-to-end registration	BLOCKED — The case cannot reach registration because the required login step fails. Record the login blocker and evidence; do not claim a registration failure that was never executed.

Part 4 — Defect Reporting and Final Quality Judgment (15 marks)

Investigate every FAILED or BLOCKED test before deciding whether it represents a defect. Check the test setup, data, environment, preconditions, and expected result; reproduce the problem where possible; and collect evidence. Log only confirmed, reproducible implementation defects in Jira. A test status and a defect are not the same thing, and not every failure or blocker should become a Jira bug.

Each Jira defect must contain:

- A short, specific title and the affected environment/build.
- Preconditions, minimal reproduction steps, expected result, and actual result.
- Reproducibility, severity, priority, supporting evidence, and the related test-case ID.
- A suitable workflow status, if used in the team's Jira project.

Final quality judgment (300–400 words):

State what the combined evidence supports, what remains unsupported, how AI-introduced assumptions affected confidence, and whether the evaluated 10-requirement scope can reasonably be considered acceptable. Your conclusion must be limited to the scope and evidence actually evaluated.

Submission Checklist

- Pair details and confirmed SRS selection from your section's Google Sheet tab.
- One report containing the 7 FR + 3 NFR scope table, AI assumptions, testing records, traceability, and the 300–400 word final judgment.
- Frozen baseline source code, setup/run instructions, and brief AI-assisted development record.
- Complete SonarQube report/evidence produced from the entire frozen codebase, plus interpretation of approximately five meaningful findings.
- Evidence for evaluation of all 3 selected NFRs.
- 12–15 executed test cases, including at least 2 boundary cases, at least 2 invalid/error cases, at least 3 manually executed system-level cases, and at least 2 non-trivial cases genuinely marked FAILED or BLOCKED.
- Jira defect evidence/export for confirmed, reproducible defects.

Demo/Viva Requirement

Important deduction rule: Submission of files does not guarantee full marks. During the demo/viva, students must answer questions about the topics covered, the selected requirements, the code, the SonarQube report, the tests, the Jira defects, and the decisions made. Failure to explain these may result in deductions, regardless of what was submitted. 'We forgot during the demo' is not an acceptable excuse. Each member of the pair should understand the complete submission.

Course Learning Outcomes

This assignment contributes to the following Course Learning Outcomes (CLOs). The evaluation rubric identifies the CLO assessed by each component.

CLO	Description and cognitive level
CLO1	Explain fundamental concepts, practices, and processes of Software Quality Engineering, including quality requirements, measurement, metrics, risk, and responsible use of generative AI. (K2 – Understand)
CLO2	Apply appropriate structural, functional, static, regression, API, and non-functional testing techniques and tools to develop effective test cases and test suites. (K3 – Apply)
CLO3	Analyze requirements, software risks, defects, quality metrics, and test evidence across the software development lifecycle to derive appropriate test strategies and quality decisions. (K4 – Analyze)

Evaluation Rubric

Component	What is being assessed	Marks	CLO mapping
Requirement scope and AI assumptions	Correct 70/30 FR–NFR selection; CRUD rule compliance; meaningful rationale/risk; material AI assumptions identified and defensibly supported, justified, or marked unsupported.	30	CLO1
AI-generated GUI baseline	Runnable GUI implements the selected scope and exposes observable behaviour; frozen baseline, run instructions, and AI-assisted development record supplied.	10	CLO2
SonarQube report and NFR evidence	Complete codebase run by the pair; authentic report/evidence submitted; meaningful findings interpreted; all 3 NFRs evaluated using relevant methods; limitations recognized.	15	CLO2
FR testing and traceability	Conditions derived from all 7 FRs; 12–15 defensible tests executed; required boundary, invalid/error, manual system-level, and genuine FAILED/BLOCKED cases included; complete traceability maintained.	30	CLO2
Defect reporting and final judgment	Failures/blockers investigated; only confirmed reproducible defects logged in Jira with evidence; final conclusion is scoped, supported, and accounts for assumptions and limitations.	15	CLO3

The demo/viva deduction rule applies across all rubric components. Marks may be reduced when submitted evidence cannot be explained or defended by the students.